Lab: Building Planet Cards with React (and Removing Them)

Objective

Build a React app that displays a card for each planet in the solar system, then add interaction so the user can remove any planet from the page.

You will practice: components, props, `map()` with keys, `useState`, and event handlers.

Starter files

File	Purpose
`index.html`	Loads React, ReactDOM and Babel from CDN, and `app.js`. Has `<div id="root">`. **Do not change.**
`app.js`	Contains the `planets` array. Write your code below the marked line.
`app.css`	Styles for `.container` and `.card`.
`html-template.html`	Example of the HTML structure each card should produce.
`img/`	Planet images.

Open `index.html` in a browser (or use a Live Server) to view your work.

Part 1: Display the planets

1. Create a `Planet` component that receives a planet's data via props and renders a card following `html-template.html`:
    - image (with `alt` text), name (`<h2>`), description (`<p>`)
    - a "Planet Profile" (`<h3>`) with a `<ul>` listing **Diameter** and **Moons**
    - Use `className`, not `class`.
2. Create a `PlanetList` (container) component that loops over the `planets` array with `map()` and renders one `Planet` per item. Use `id` as the key.
3. Render `PlanetList` into `#root` using `ReactDOM.createRoot(...)`.

Checkpoint: all 8 planets appear as cards. Your page should look like this (the red Remove button is added in Part 2):

All eight planet cards in a responsive grid

Each card contains the image, the name, the description and the planet profile list:

```
┌──────────────────────────┐
│ [ planet image on black] │
│ Mercury                  │   <h2>
│ Mercury is the closest…  │   <p>
│ Planet Profile           │   <h3>
│  • Diameter: 3,031.67 mi │   <ul>
│  • Moons: none           │
│ [ Remove ]               │   <button> (Part 2)
└──────────────────────────┘
```

Part 2: Remove a planet

1. In `PlanetList`, store the planets in state:
   ```
   const [planetList, setPlanetList] = React.useState(planets);
   ```
   and map over `planetList` instead of `planets`.
2. Write a `removePlanet(id)` function in `PlanetList` that updates state with a **new array** that excludes the planet with that id (use `filter()`). Never modify the existing array with `splice`/`push`.
3. Pass `removePlanet` to each `Planet` as a prop (e.g. `onRemove`).
4. In `Planet`, add a **Remove** button. When clicked, call `onRemove` with that planet's `id`.
5. Style the button in `app.css` (your choice; add a hover state).

Checkpoint: clicking Remove makes that card disappear; the others remain, and the grid re-flows to fill the gap.

Part 3: Extras (optional)

- Show a message such as "No planets left!" when the list is empty.
- Display the number of remaining planets in a heading.
- Add a **Reset** button that restores all planets.
- Ask for confirmation (`window.confirm`) before removing.

Submission

Submit your `app.js` and `app.css` (zipped with `index.html` and `img/`).

Hints

- Props: `function Planet({ planet, onRemove }) { ... }`
- Handler with an argument: `onClick={() => onRemove(planet.id)}`
- State updates: `setPlanetList(planetList.filter(p => p.id !== id))`

The following image show the renders of the app.

